Meeting 15/04/2026

Summary

Action Items & Next Steps

- □ Resolve public IP exposure: configure MetalLB LoadBalancer pool on the college LAN, set up DNAT and SNAT via iptables on the Linux gateway; if ISP gives DHCP use MASQUERADE; if the router is a Cisco L3 device use static NAT
- □ Configure SSH access for students via port-forward DNAT (public port 2222 to internal node:22) or set up a jump host inside the private network
- □ CI pipeline: enforce CI on merge with linting and SonarQube; version releases as 1.0.0 and 1.0.1 on the dev branch
- □ Define and apply PVC data persistence strategy for all stateful workloads: MongoDB replica set, LocalStack S3, Sonatype Nexus, Keycloak, Loki, Jaeger, Prometheus, Grafana, and Kafka
- □ Implement full user registration and onboarding flow: form POST to Kafka, email notification, admin approval via Keycloak admin API
- □ Implement login flow: bearer token and session token issued by Keycloak, JWT claims for RBAC, session timeout and forced logout
- □ Define MongoDB sharding and replica set strategy; implement backup and recovery via mongodump to LocalStack S3
- □ Organise the project documentation into six chapters: Development, Security, DevOps and CloudOps, Observability, Operations, and Quality
- □ Define the Quality chapter scope: load testing, scale testing, and functional testing with Apdex scoring

Concepts Discussed

Public IP and NAT Configuration

- Problem: RKE2 cluster sits on a private /24 network (192.168.1.5); services need to be reachable from the internet and students need SSH access
- Solution overview: MetalLB allocates a pool of LAN IPs (e.g. 192.168.1.240 to 192.168.1.250) and assigns them to LoadBalancer services; the edge gateway translates public IP traffic to these MetalLB IPs via DNAT
- Linux gateway with static public IP: enable ip_forward, add DNAT PREROUTING rules (public IP port 80 to MetalLB IP port 80), and MASQUERADE on the outbound interface for return traffic
- Linux gateway with dynamic DHCP public IP (ISP like Airtel): replace SNAT with MASQUERADE so the outbound rule always uses the current public IP; add hairpin NAT for internal clients
- Cisco L3 switch or router with IOS: use static NAT commands (ip nat inside source static tcp) to map MetalLB IPs and the SSH port to the public IP; assign ip nat inside and ip nat outside on the relevant interfaces
- SSH for students: DNAT public port 2222 to an internal node port 22, or provision a jump host inside the private network for auditing and access control

CI Pipeline and Versioning

- CI on merge enforced: linting and SonarQube quality gate must pass before a PR merges to the dev branch
- Release versioning: dev branch tracks incremental releases (1.0.0, 1.0.1); each successful pipeline run tags and publishes a versioned artifact to Sonatype Nexus

Kubernetes Data Persistence Strategy (PVC)

- All stateful workloads run as StatefulSets with explicit PersistentVolumeClaims; no ephemeral storage for production data
- MongoDB replica set: each replica pod gets its own PVC backed by local-path or Longhorn storage; replica-set backups run from the secondary to avoid impacting the primary
- LocalStack S3, Sonatype Nexus, Keycloak: each uses a dedicated PVC; Nexus blob store and Keycloak realm data must survive pod restarts without data loss
- Observability stack (Loki, Jaeger, Prometheus, Grafana): PVCs sized for log and metric retention windows; Prometheus uses a StatefulSet with a dedicated PVC for TSDB
- Kafka: each broker pod in the StatefulSet gets its own PVC for log segment storage; topic offsets and consumer group state are persisted to the PVC, not ephemeral storage
- Sensitive data handling: Kubernetes Secrets (base64-encoded, RBAC-restricted) for credentials and tokens; control plane etcd data backed up via etcdctl snapshot on a schedule

User Registration, Onboarding, and Auth Flow

- New user registration: form POST from the frontend container publishes an event to Kafka (or SQS); a notification service sends a confirmation email; admin reviews the submission on the admin page
- Admin approval: admin pulls the pending event from the message bus, approves or rejects; on approval the Keycloak admin API creates the user account and assigns the appropriate role
- Login flow: user authenticates against Keycloak; Keycloak issues a bearer token (short-lived access token) and a session token stored in an httpOnly cookie; JWT claims carry role and entitlement information for RBAC decisions
- Session management: idle timeout and forced logout configured in Keycloak; token refresh handled by the frontend; revocation propagated via Keycloak session invalidation

MongoDB Sharding, Backup, and Recovery

- Replica sets provide high availability; manual sharding across multiple containers used if horizontal scaling is needed; replica set topology defined in the StatefulSet with a headless service for stable DNS
- Backup strategy: mongodump exports to BSON and writes to LocalStack S3 via the existing Boto3 REST API; mongorestore handles recovery; backups run from a secondary replica via a Kubernetes CronJob
- Filesystem snapshots used for large datasets; replica set lag must be minimal before a snapshot is taken to ensure consistency

Documentation Chapter Structure

- The project documentation is organised into six chapters covering the full lifecycle of the platform:
  - Development: API design, service code, OpenAPI specs, and database schema
  - Security: zero-trust model, Keycloak RBAC, Istio mTLS, WAF, and DPI

- DevOps and CloudOps: CI/CD pipelines, Helm charts, Terraform, and multi-cluster management
- Observability: Loki, Jaeger, Prometheus, Grafana, and alerting
- Operations: backup and recovery, runbooks, incident response, and SLA monitoring
- Quality: load testing (k6/JMeter), scale testing, functional testing, and Apdex scoring